

Generative AI models

Many models to choose from...

- Differ in training data
- Differ on range of tasks and instructions trained
- Differ on size of context window (window of data)
 - Focus of course mostly on language models (the most relevant for security)

Text-to-Text	Text-to-Image	Image-to-Text	Image-to-3D	Image or Video-to-3D
GPT Gemini Claude LLaMA	Stable Diffusion Midjourney DALL-E Imagen Nano Banana	GPT Gemini Claude LLaVA	Magic3D DreamFusion GET3D	CSM AI NeRF
Text-to-Audio	Text-to-Code	Image-to-Science	Text-to-video	Audio-to-text
AudioGen ElevenLabs Jukebox	GPT AlphaCode Code LLaMA	Minerva Galactica BioGPT	Runway Cuebic Phenaki	Whisper Granite VoiceBase

Large language models

- What is a language model?
 - "A probability distribution over sequences of tokens"
 - From CS 443/543: Large-Language Models
 - Model assigns each sequence a probability telling us how "good" it is based on what it has seen in training
 - Generates most probable sequence
- Many models (GPT, Gemini, Mistral, Claude, Llama)
- Many different datasets and instructions they are trained on
 - CommonCrawl, Books, Wikipedia, Social media conversations, Github, News, arxiv, StackExchange, StackOverflow, etc.

LLMs

- Two main tasks for text-to-text models
 - Natural language understanding (processing) or NLP
 - Natural language generation or NLG

```
from langchain_google_genai import ChatGoogleGenerativeAI  
llm = ChatGoogleGenerativeAI(model="gemini-1.5-pro")  
response = llm.invoke("Write me a haiku about Portland State University")  
print(response.content)
```

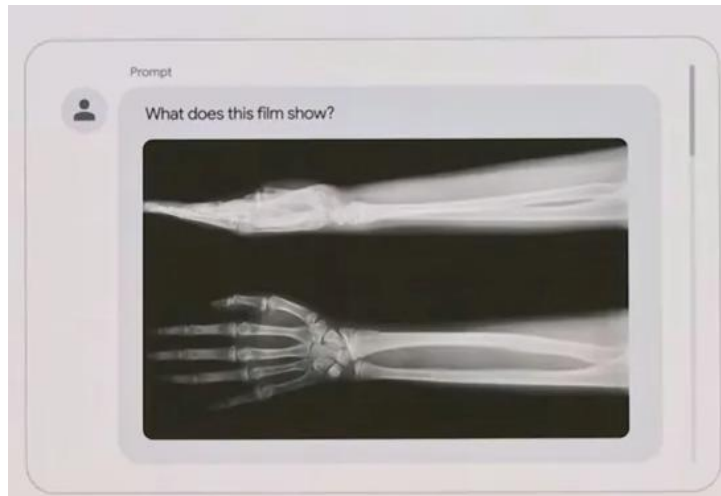
PSU, a haven
Learning, art, and green spaces
Portland's urban heart

But increasingly...

- Multiple specialized models tuned and targeted towards...
 - Specific tasks such as reasoning (OpenAI Strawberry)



- Specific contexts such as healthcare



- Specific tasks such as reverse engineering and malware analysis



Introducing VirusTotal Code Insight:
Empowering threat analysis with generative AI

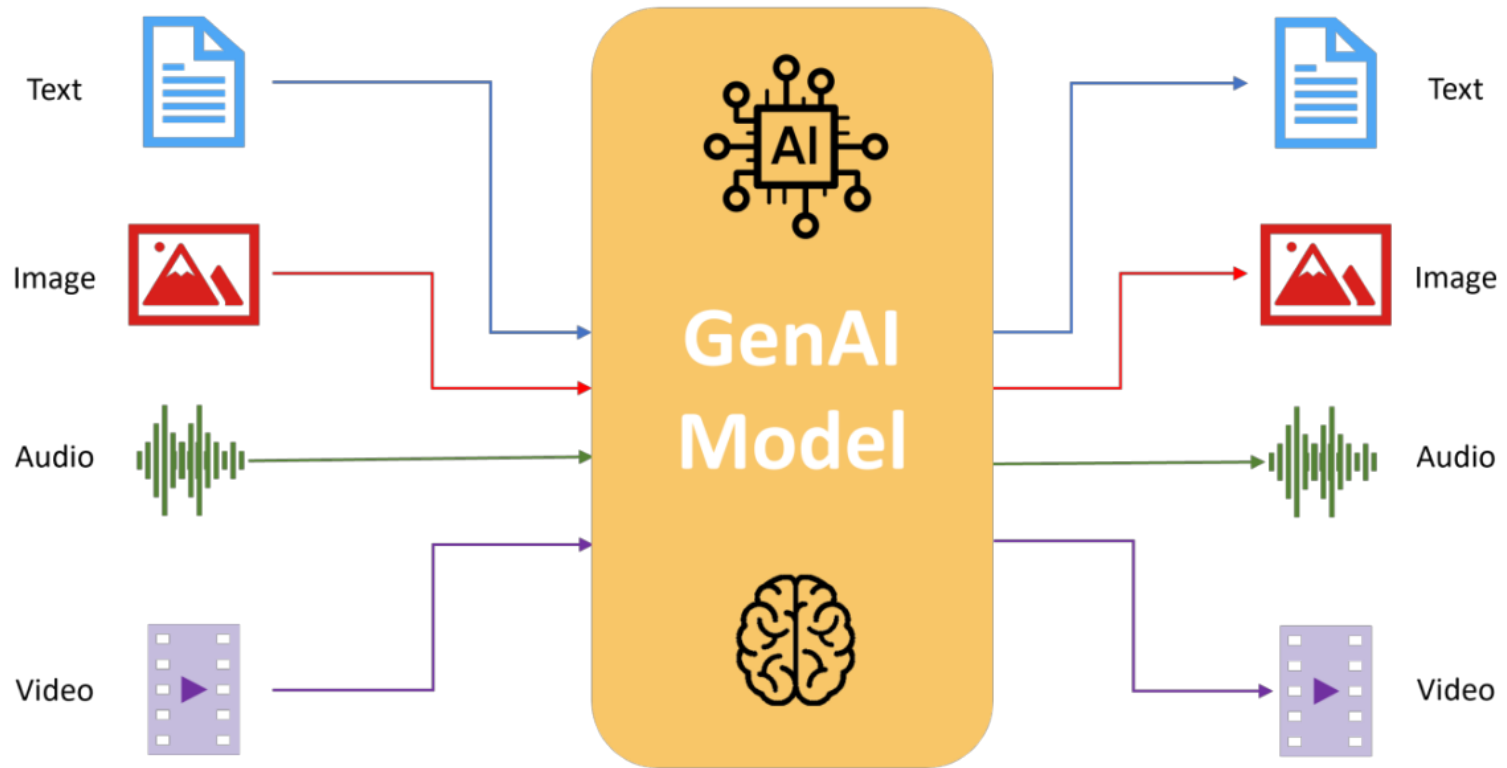
MONDAY, APRIL 24, 2023 | [BERNARDO.QUINTERO](#)

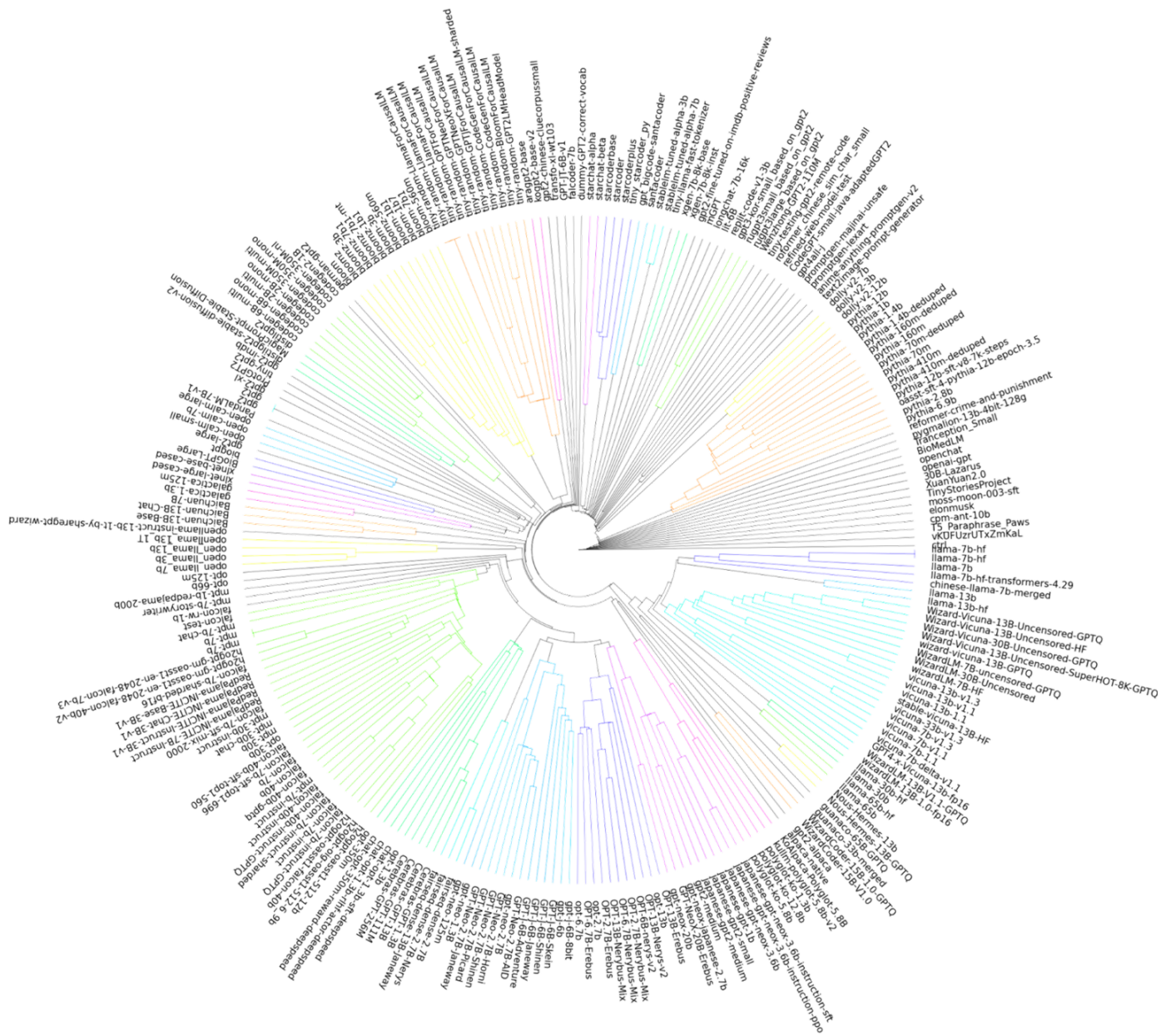
Threat Intelligence

From Assistant to Analyst: The Power of Gemini 1.5 Pro for Malware Analysis

April 29, 2024

- Multimodal models
 - Tuned and targeted towards multiple content formats





When should one use an LLM?

- To replace processes that automation has not already solved
 - Automation that already solves the problem is likely cheaper and should be used instead
 - Later in the course...LLMs orchestrating automated tools
- To replace processes done by humans
 - Must ultimately reduce costs with solving a particular problem manually

Models and their parameters

Using models

- Back to simple example
 - Model used typically as a "black box"

```
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(model="gemini...")
response = llm.invoke("Write me a haiku about Portland State University")
print(response.content)
```



```
PSU, a haven
Learning, art, and green spaces
Portland's urban heart
```

- But, programmatically one can change the behavior of the model

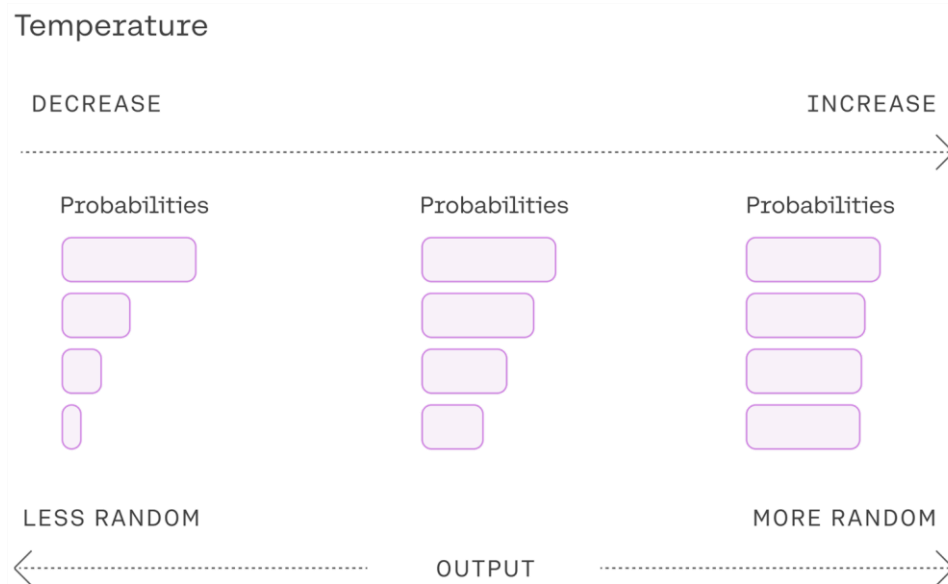
Models and their parameters

- Controlling model behavior
 - Temperature
 - Top-k, Top-p
 - Max Tokens/Length
 - Candidate Count
 - Streaming
 - Safety settings
- Example (Gemini)

```
llm = ChatGoogleGenerativeAI(model="gemini...",
    max_output_tokens = 200,
    temperature = 0.7,
    candidate_count = 2,
    top_k = 5,
    stream=True,
    safety_settings = {
        HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT:
            HarmBlockThreshold.BLOCK_ONLY_HIGH,
    }
)
response = llm.invoke("Write me a haiku about Portland State University")
```

Temperature

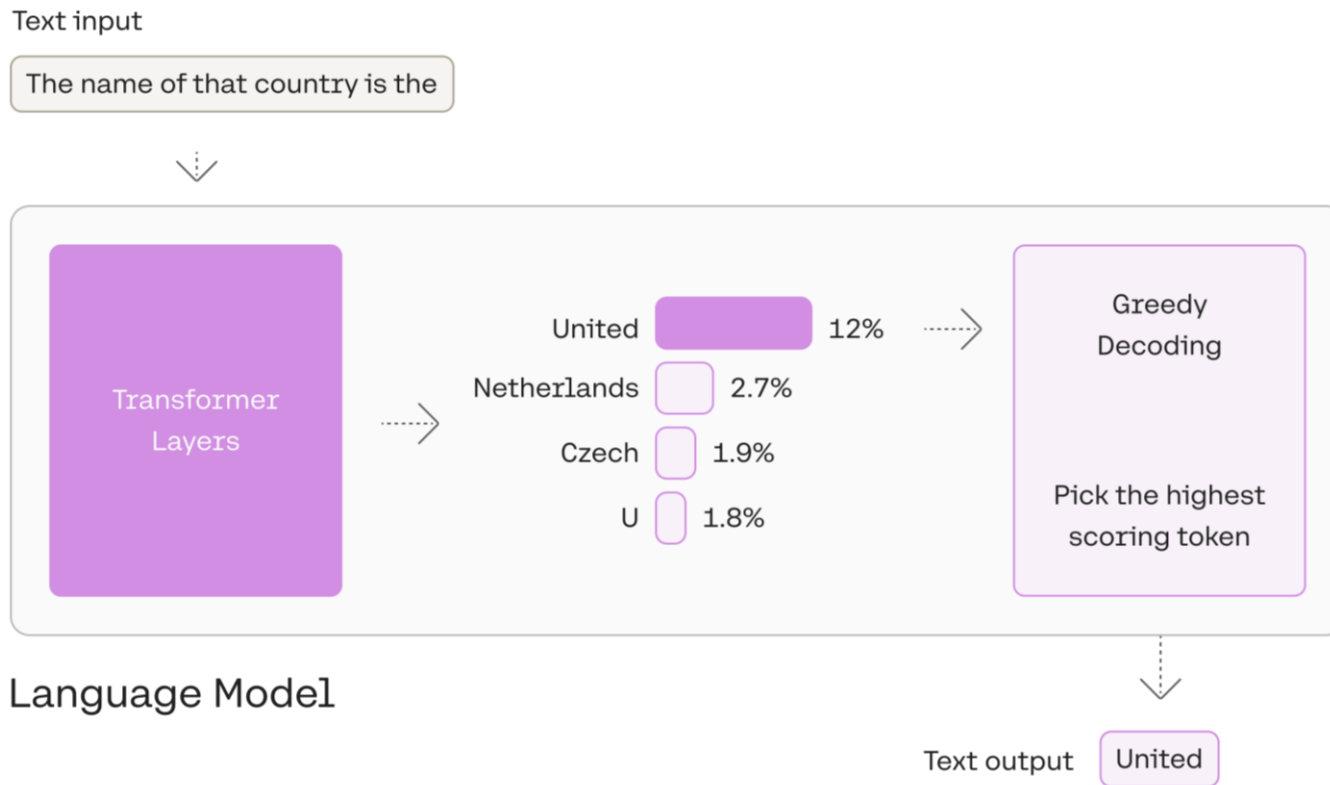
- Controls the randomness of the response
 - Increase temperature (0 to 1) == increase randomness
 - Example
 - Deceptive content generation might need high temperature
 - Reasoning tasks might need low temperature



```
llm = ChatGoogleGenerativeAI(model="gemini...")
response = llm.invoke("Write me a haiku about Portland State University",
                      temperature=0.8)
```

Top-k and top-p

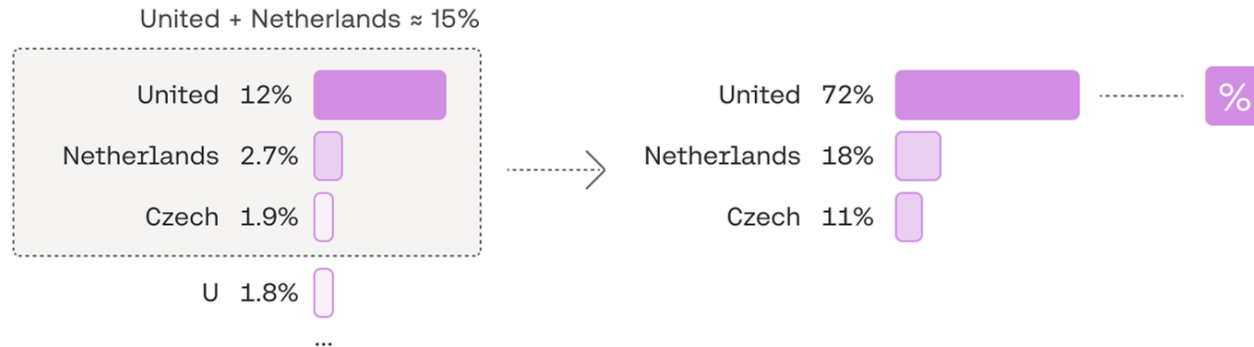
- Related to temperature
 - How to choose candidate outputs from a probability distribution?
- Greedy algorithm picks the top candidate (temperature == 0)



- Top-k picks from the most likely k (e.g. 3) (GPT-2)

1. Consider only the top 3 tokens.
Ignore all others.

2. Sample from them based on
their likelihood scores.

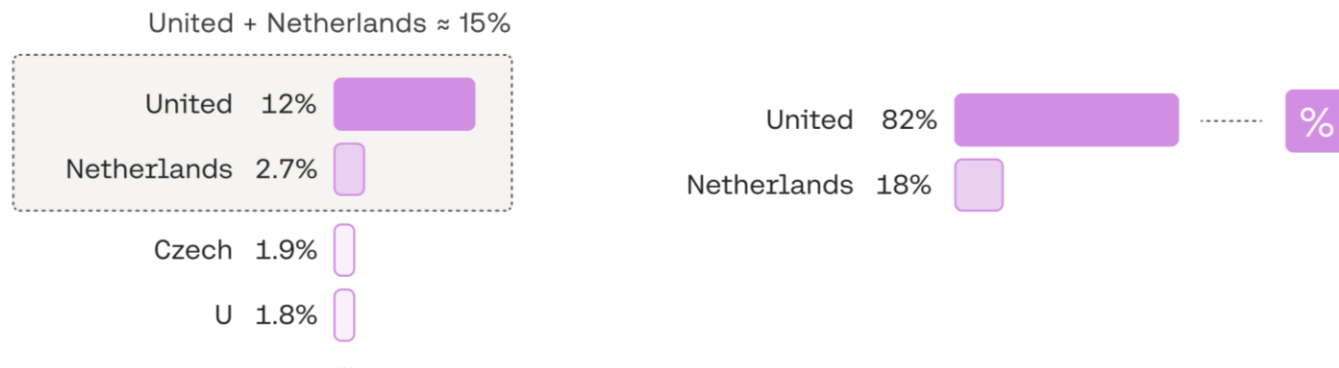


- Top-p (nucleus sampling) picks from the most likely candidates whose probabilities add up to p (e.g. 15)

- Larger p means more candidates

1. Consider only the top tokens whose
likelihoods add up to 15%. Ignore all others.

2. Sample from them based on their
likelihood scores.

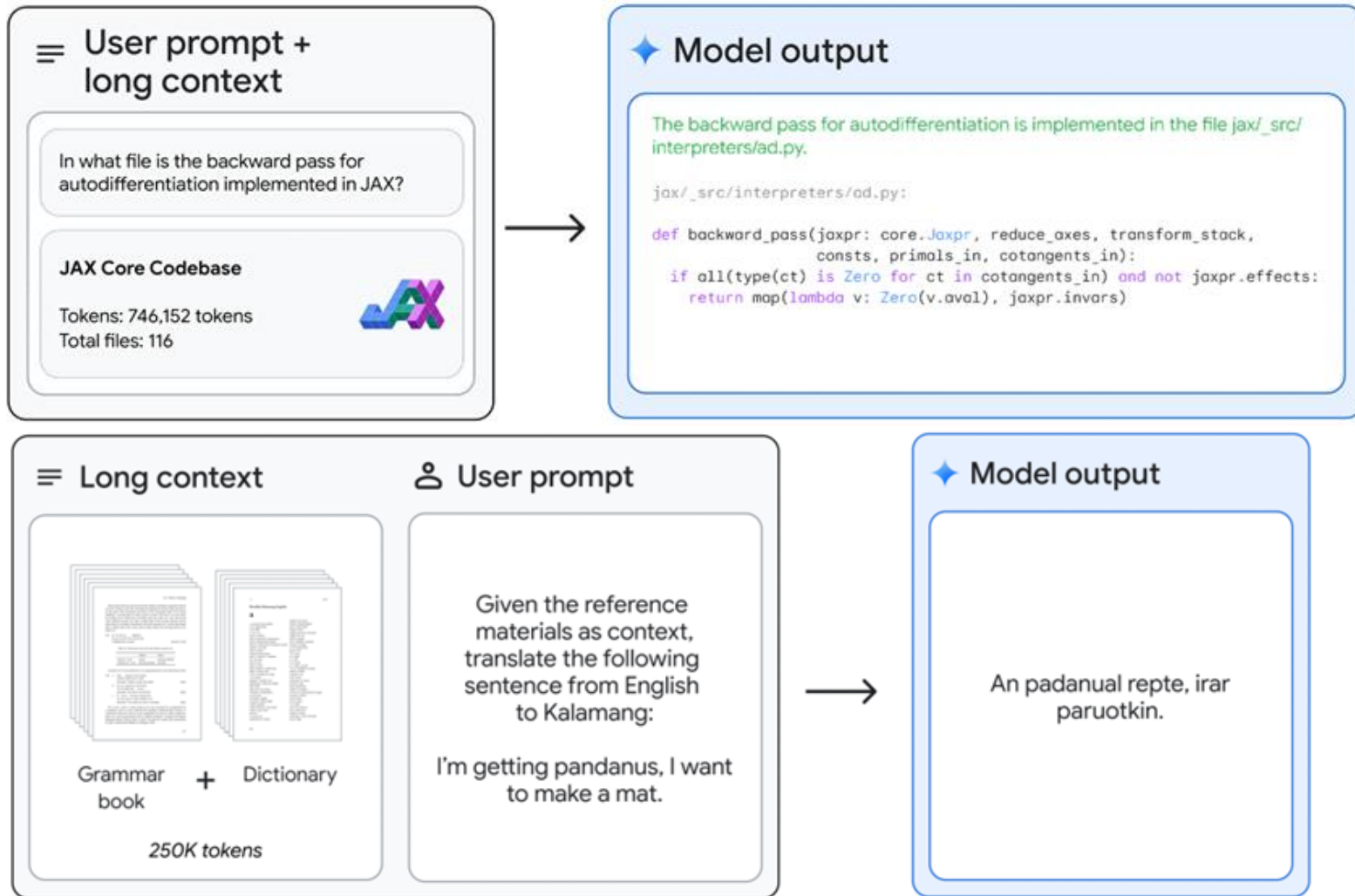


Other parameters

- Candidate Count
 - Number of response variations to return
- Streaming
 - Whether to return results as they are generated versus waiting until generation is complete
- Safety settings
 - Ensuring output is not harmful or offensive
- Max Tokens
 - Limit the length of input
 - Limit the length of output response

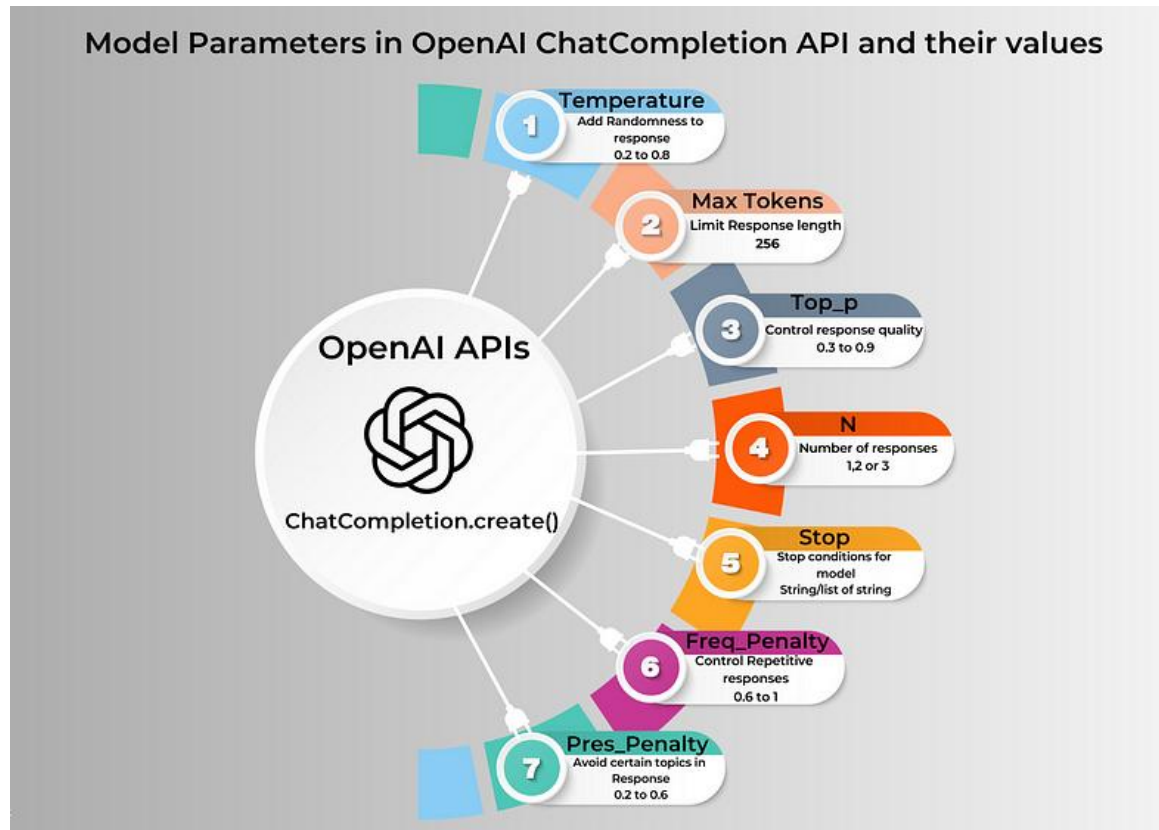
Model context window size

- Maximum number of tokens a model can take as input
 - Can a model handle large input (e.g. needle in haystack)?



Compatibility

- Many models to choose from
 - Want to be able to move from one model to another easily
- Most models support common subset derived from OpenAI's models (interface compatibility)



How to build an OpenAI-compatible API

Create a server to replicate OpenAI's Chat Completions API, enabling any LLM to integrate with tools written for the OpenAI API



Saar Berkovich · [Follow](#)

Published in Towards Data Science · 6 min read · 6 days ago

```
from pydantic import BaseModel

class ChatMessage(BaseModel):
    role: str
    content: str

class ChatCompletionRequest(BaseModel):
    model: str = "mock-gpt-model"
    messages: List[ChatMessage]
    max_tokens: Optional[int] = 512
    temperature: Optional[float] = 0.1
    stream: Optional[bool] = False
```

- Allows one to do this (in this week's labs)...

```
from langchain.model_laboratory import ModelLaboratory
from langchain_community.llms import HuggingFaceHub, HuggingFaceEndpoint
from langchain_community.llms import Ollama
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
```

```
llms = [
    Ollama(model="llama3", base_url="http://35.236.12.22:11434"),
    HuggingFaceEndpoint(repo_id="mistralai/Mistral-7B-Instruct-v0.2",),
    HuggingFaceHub(repo_id="google/gemma-2b"),
    ChatOpenAI(model="gpt-4o", temperature=0),
    ChatGoogleGenerativeAI(model="gemini-1.5-pro"),
    ChatAnthropic(model="claude-3-sonnet-20240229")
]
model_lab = ModelLaboratory.from_llms(llms)
model_lab.compare("Write me a haiku about Portland State University")
```

Basic prompting

<https://www.promptingguide.ai/>

Elements of a prompt

"Produce a single integer that answers the following question using the information included.

Question: How many credits does it take to graduate?

Information: To graduate, one must pass all required courses (90 credits) as well as take all general education courses (90 credits).

Answer: "

- Instruction or task
- Input data
 - Input or question we want a response for
- Context
 - External information model should use to generate response
- Output indicator
 - Type and format of output

Strategy #1: Delineate clearly

- Lack of clarity between instructions, context, and input data confuses model
 - Example
 - Utilize symbols and special formatting (e.g. ### or """)

```
### Instruction ###  
Translate the text below to Spanish:  
  
Text: "hello!"
```

Strategy #2: Be specific

- Be detailed and descriptive, while avoiding unnecessary description
 - Specific output length, audience, details to include/exclude, and format
 - Example: Specifying output format

Provide the current time in a JSON object that contains keys for month, day, year, hour, minute, and second. Use Pacific Standard Time. Do not respond with any other text than the JSON object



```
{  
  "month": 4,  
  "day": 3,  
  "year": 2024,  
  "hour": 13,  
  "minute": 35,  
  "second": 22  
}
```


Strategy #3: Be precise

- Avoid subjective interpretation
 - Subjective description

Explain the concept prompt engineering. Keep the explanation short, only a few sentences, and don't be too descriptive.

- Precise description

Use 2-3 sentences to explain the concept of prompt engineering to a high school student.

Strategy #4: Use positive statements

The following is an agent that recommends movies to a customer. DO NOT ASK FOR INTERESTS. DO NOT ASK FOR PERSONAL INFORMATION.

Customer: Please recommend a movie based on my interests.

Agent:



Sure, I can recommend a movie based on your interests. What kind of movie would you like to watch? Do you prefer action, comedy, romance, or something else?

The following is an agent that recommends movies to a customer. The agent is responsible to recommend a movie from the top global trending movies. It should refrain from asking users for their preferences and avoid asking for personal information. If the agent doesn't have a movie to recommend, it should respond "Sorry, couldn't find a movie to recommend today.".

Customer: Please recommend a movie based on my interests.

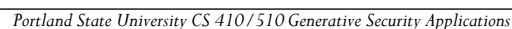
Agent:



Sorry, I don't have any information about your interests. However, here's a list of the top global trending movies right now: [list of movies]. I hope you find something you like!

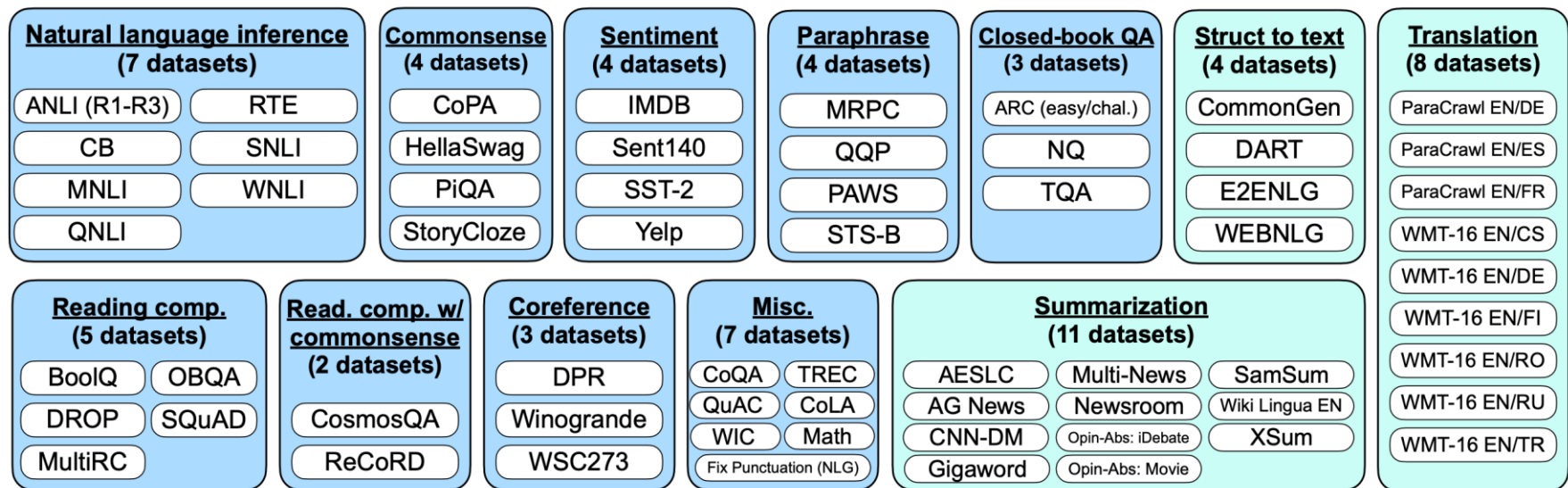
Models

- Categorization of 1600+ NLP tasks
 - Use a model best suited for your task



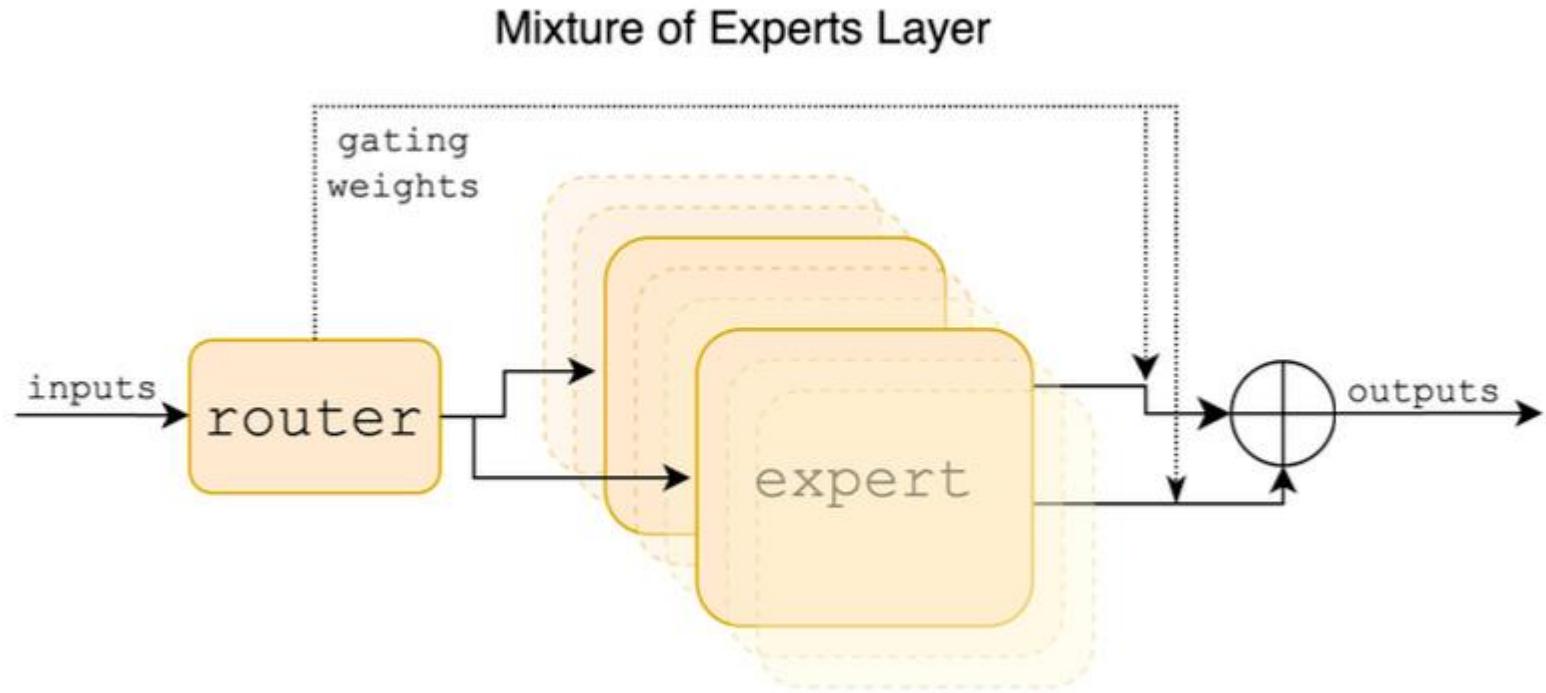
Instruction tuned operation

- Internally, many models tuned to specific instructions to help it specialize in solving specific tasks (Wei 2022)



- Many are adopting a "Mixture of Experts" architecture with per-task neural networks to complete tasks
 - NLU on input task description decides which expert to ask to perform NLG task

- Example
 - Mixtral



- How can you determine which model to use for your task?

Task benchmarking

Spaces HuggingFaceH4/open_llm_leaderboard 8.82k Running on CPU UPGRADE App Files Community 652

Open LLM Leaderboard

LLM Benchmark Metrics through time About FAQ Submit

T	Model	Average	ARC	HellaSwag	MMLU	TruthfulQA	Winogrande	GSM8K
	davidkim205/Rhea-72b-v0.5	81.22	79.78	91.15	77.95	74.5	87.85	76.12
	davidkim205/Rhea-72b-v0.4	81.09	78.5	90.75	78.01	73.91	86.74	78.62
	MTSAIR/MultiVerse_70B	81	78.67	89.77	78.22	75.18	87.53	76.65
	binbi/Ein-72B-v0.1	80.99	76.45	89.43	77.14	78.09	84.77	80.06
	MTSAIR/MultiVerse_70B	80.98	78.58	89.74	78.27	75.09	87.37	76.8
	davidkim205/Rhea-72b-v0.2	80.95	77.56	90.84	77.98	74.5	86.35	78.47

- HuggingFace leaderboard benchmark metrics
 - ARC-C: Grade-school science questions.
 - HellaSwag : Common sense reasoning.
 - MMLU: multitask accuracy, problem solving ability.
 - WinoGrande: commonsense reasoning around pronoun resolution.
 - Natural2Code: python code generation.
 - WMT23: machine translation.
 - GSM8K: grade-school math.
 - NaturalQuestions: reading comprehension and question answering.
 - TruthfulQA: truthfulness in generating answers to questions.

DeepMind benchmarks (12/2025)

- For Gemini 3 [Flash](#)
 - Reasoning

Benchmark	Description		Gemini 3 Flash Thinking	Gemini 3 Pro Thinking	Claude Sonnet 4.5 Thinking	GPT-5.2 Extra high
Humanity's Last Exam	Academic reasoning (full set, text + MM)	No tools	33.7%	37.5%	13.7%	34.5%
		With search and code execution	43.5%	45.8%	—	45.5%
ARC-AGI-2	Visual reasoning puzzles	ARC Prize Verified	33.6%	31.1%	13.6%	52.9%
GPQA Diamond	Scientific knowledge	No tools	90.4%	91.9%	83.4%	92.4%
AIME 2025	Mathematics	No tools	95.2%	95.0%	87.0%	100%
		With code execution	99.7%	100%	100%	—
MMMU-Pro	Multimodal understanding and reasoning		81.2%	81.0%	68.0%	79.5%
Global PIQA	Commonsense reasoning across 100 Languages and Cultures		92.8%	93.4%	90.1%	91.2%

- Information extraction and synthesis

Benchmark	Description		Gemini 3 Flash Thinking	Gemini 3 Pro Thinking	Claude Sonnet 4.5 Thinking	GPT-5.2 Extra high
ScreenSpot-Pro	Screen understanding	No tools unless specified	69.1%	72.7%	36.2%	86.3% with python
CharXiv Reasoning	Information synthesis from complex charts	No tools	80.3%	81.4%	68.5%	82.1%
OmniDocBench 1.5	OCR	Overall Edit Distance, lower is better	0.121	0.115	0.145	0.143
Video-MMMU	Knowledge acquisition from videos		86.9%	87.6%	77.8%	85.9%

- Code generation

Benchmark	Description		Gemini 3 Flash Thinking	Gemini 3 Pro Thinking	Claude Sonnet 4.5 Thinking	GPT-5.2 Extra high
LiveCodeBench Pro	Competitive coding problems from Codeforces, ICPC, and IOI	Elo rating, higher is better	2316	2439	1418	2393
Terminal-bench 2.0	Agentic terminal coding	Terminus-2 harness	47.6%	54.2%	42.8%	—
SWE-bench Verified	Agentic coding	Single attempt	78.0%	76.2%	77.2%	80.0%

- Agentic tasks (more later)

Benchmark	Description	Gemini 3 Flash Thinking	Gemini 3 Pro Thinking	Claude Sonnet 4.5 Thinking	GPT-5.2 Extra high
τ2-bench	Agentic tool use	90.2%	90.7%	87.2%	—
Toolathlon	Long horizon real-world software tasks	49.4%	36.4%	38.9%	46.3%
MCP Atlas	Multi-step workflows using MCP	57.4%	54.1%	43.8%	60.6%

- Factuality, Q&A

Benchmark	Description	Gemini 3 Flash Thinking	Gemini 3 Pro Thinking	Claude Sonnet 4.5 Thinking	GPT-5.2 Extra high
FACTS Benchmark Suite	Factuality benchmark across grounding, parametric, search, and MM	61.9%	70.5%	48.9%	61.4%
SimpleQA Verified	Parametric knowledge	68.7%	72.1%	29.3%	38.0%
MMMLU	Multilingual Q&A	91.8%	91.8%	89.1%	89.6%

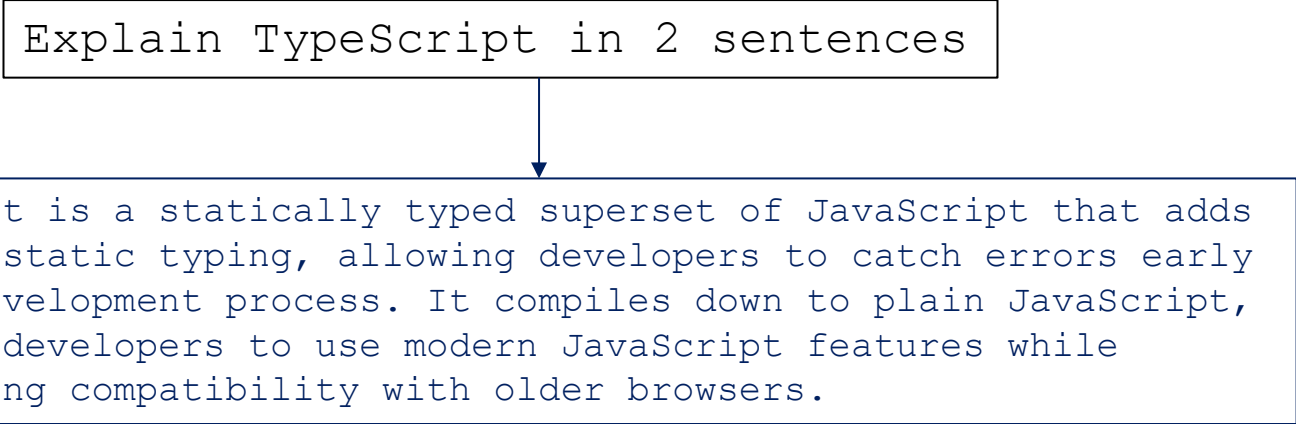
- Long context performance

Benchmark	Description		Gemini 3 Flash Thinking	Gemini 3 Pro Thinking	Claude Sonnet 4.5 Thinking	GPT-5.2 Extra high
MRCR v2 (8-needle)	Long context performance	128k (average)	67.2%	77.0%	47.1%	81.9%
		1M (pointwise)	22.1%	26.3%	not supported	not supported

Model tasks

Text generation

Explain TypeScript in 2 sentences



TypeScript is a statically typed superset of JavaScript that adds optional static typing, allowing developers to catch errors early in the development process. It compiles down to plain JavaScript, enabling developers to use modern JavaScript features while maintaining compatibility with older browsers.

Code generation

- Copilot

Ask the user for their name and say "Hello"

↓

```
let name = prompt("What is your name?");  
console.log(`Hello, ${name}!`);
```

- Any issues?

- String literal allows user to execute arbitrary Javascript code!
- Effectively a Javascript `eval()`

Modify the program to remove any potential of code execution

↓

```
const userName = prompt("What is your name?");  
  
if (/^[a-zA-Z0-9]+$/.test(userName)) {  
    alert(`Hello ${userName}!`);  
} else {  
    alert("Please enter a valid name.");  
}
```

- Database example

```
"""
```

```
Table departments, columns = [DepartmentId, DepartmentName]
```

```
Table students, columns = [DepartmentId, StudentId, StudentName]
```

Given the schema above, create a MySQL query for all students in the Computer Science Department.

```
"""
```



```
SELECT StudentId, StudentName
```

```
FROM students
```

```
WHERE DepartmentId IN (SELECT DepartmentId FROM departments WHERE  
DepartmentName = 'Computer Science');
```

Text summarization

In computer science, type safety and type soundness are the extent to which a programming language discourages or prevents type errors. Type safety is sometimes alternatively considered to be a property of facilities of a computer language; that is, some facilities are type-safe and their usage will not result in type errors, while other facilities in the same language may be type-unsafe and a program using them may encounter type errors. The behaviors classified as type errors by a given programming language are usually those that result from attempts to perform operations on values that are not of the appropriate data type, e.g., adding a string to an integer when there's no definition on how to handle this case. This classification is partly based on opinion.

Explain the above in one sentence:



Type safety and type soundness in computer science refer to the degree to which a programming language prevents type errors, with type-safe facilities ensuring error-free usage while type-unsafe ones may lead to such errors, typically arising from inappropriate operations on incompatible data types.

Code summarization

Write a one-line summary of what this code does.

```
"""  
''.join(chr(x) for x in [72, 101, 108, 108, 111, 32, 87, 111, 114, 108,  
100])  
"""
```



This code constructs the string "Hello World" by converting a list of ASCII values representing each character into their corresponding characters and joining them together.

Write a one-line summary of what this code does.

```
"""  
const express = require('express');  
const app = express();  
app.get('/', (req, res) => {  
  const user = req.query.user;  
  res.send(`<h1>Hello ${user}</h1>`);  
});  
"""
```



This code creates a Node.js web server using the Express framework that listens for GET requests on the root path ("/"). When a request is received, it retrieves the value of the "user" parameter from the URL query and responds with HTML that includes a greeting message containing the value of the "user" parameter.

Text translation

Translate the following into Spanish.

"It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness"



"Era el mejor de los tiempos, era el peor de los tiempos, era la edad de la sabiduría, era la edad de la insensatez."

Code translation

Translate the code below to TypeScript

```
""
const express = require('express');
const app = express();
app.get('/', (req, res) => {
  const user = req.query.user;
  res.send(`<h1>Hello ${user}</h1>`);
});
""
```



```
import express, { Request, Response } from 'express';
const app = express();

app.get('/', (req: Request, res: Response) => {
  const user = req.query.user as string;
  res.send(`<h1>Hello ${user}</h1>`);
});
```

In this TypeScript version:

- We import the express module to provide type information for Express.
- We explicitly specify the types for the req (Request) and res (Response) parameters in the route handler using TypeScript's type annotations.
- We use the as string assertion to tell TypeScript that req.query.user is of type string, as it might be undefined if the query parameter is not provided.

Classification (e.g. sentiment analysis)

Classify the text into neutral, negative or positive.

Text: I think the food was awesome.

Sentiment:

positive

Classify the e-mail subject into malicious or benign.

Text: You've won \$1000. Click on attachment for instructions to claim

Sentiment:

malicious

Information extraction

AI (Artificial Intelligence) systems can process large amounts of data and uncover threats that human beings might overlook. This makes quick action possible, as AI can monitor network traffic, user activities, and system logs and identify abnormal actions, intrusions, and cyberattacks. A Spanish pentester with a "luijait" alias on Github recently unveiled an AI OSINT tool dubbed "DarkGPT" that helps detect leaked databases. Based on GPT-4-200K, DarkGPT is an AI (Artificial Intelligence) assistant that can run queries on databases that have been compromised. Its ability to learn and adapt is another important advantage of AI in cybersecurity.

Based on the paragraph above, what does the tool described do?



The tool described, "DarkGPT," is an AI OSINT (Open Source Intelligence) tool created by a Spanish pentester that utilizes the GPT-4-200K model to detect leaked databases by running queries on compromised databases, leveraging AI's ability to process large amounts of data and adapt to new information.

Question Answering

Answer the question based on the context below. Keep the answer short and concise. Respond "Unsure about answer" if not sure about the answer.

Context: Traditional IT network security is based on the castle-and-moat concept. In castle-and-moat security, it is hard to obtain access from outside the network, but everyone inside the network is trusted by default. The problem with this approach is that once an attacker gains access to the network, they have free rein over everything inside. Zero Trust security means that no one is trusted by default from inside or outside the network, and verification is required from everyone trying to gain access to resources on the network. This added layer of security has been shown to prevent data breaches. Studies have shown that the average cost of a single data breach is over \$3 million. Considering that figure, it should come as no surprise that many organizations are now eager to adopt a Zero Trust security policy.

Question: What does traditional IT security trust that Zero Trust security does not?

Answer:



Traditional IT security trusts everyone inside the network by default, whereas Zero Trust security does not trust anyone by default, whether inside or outside the network.

Role-based conversation

The following is a conversation with an AI research assistant. The assistant tone is technical and mathematical.

Human: Hello, who are you?

AI: Greetings! I am an AI research assistant. How can I help you today?

Human: Can you tell me about the elliptic curve cryptography used in public-key systems?

AI:

Certainly! Elliptic curve cryptography (ECC) is a cryptographic algorithm that relies on the mathematical properties of elliptic curves over finite fields. It offers strong security with smaller key sizes compared to other public-key systems, making it suitable for constrained environments like mobile devices or IoT devices. The security of ECC is based on the difficulty of solving the elliptic curve discrete logarithm problem (ECDLP), which involves finding the integer n such that $Q=nP$ for given points P and Q on the curve. This problem is believed to be computationally infeasible to solve efficiently, even for quantum computers, when properly configured parameters are used. As a result, ECC is widely used in various cryptographic protocols such as TLS, SSH, and digital signatures.

The following is a conversation with an AI research assistant. The assistant answers should be easy to understand even by primary school students.

Human: Hello, who are you?

AI: Greeting! I am an AI research assistant. How can I help you today?

Human: Can you tell me about the elliptic curve cryptography used in public-key systems?

AI:

Elliptic curve cryptography is like a special secret code that keeps things safe on the internet. It uses points on a curve to make really strong passwords that help protect your messages and information when you send them online.

Reasoning

If I have \$30 and I take a taxi ride that goes for 5 miles that costs \$0.75 a mile, how much will I have left assuming I give a \$3 tip



The cost of the ride is 5 miles * \$0.75/mile = \$3.75.
In total you will spend \$3.75 + \$3 = \$6.75 on the ride.
So you will have \$30 - \$6.75 = \$23.25 left.

So the answer is 23.25

Common reasoning tasks

- Deductive
- Inductive
- Mathematical
- Temporal
- Spatial
- Common-sense
- Causal
- Multi-hop
- Analogical (adding numbers example)

- Deductive reasoning

All humans are mortal.
Socrates is a human.
Is Socrates mortal?



Yes, based on the premises "All humans are mortal" and "Socrates is a human," we can deduce that Socrates is indeed mortal. This follows the principle of syllogism, which states that if all members of a group (in this case, humans) share a certain property (mortality), and an individual (Socrates) belongs to that group, then that individual also possesses that property. Therefore, Socrates must be mortal.

- Inductive reasoning

Given the following sequence: 2, 4, 8, 16, 32, ...
What is the next number in the sequence?



The given sequence appears to be doubling each time:

$$2 * 2 = 4$$

$$4 * 2 = 8$$

$$8 * 2 = 16$$

$$16 * 2 = 32$$

So, following this pattern, the next number in the sequence would be $32 * 2 = 64$.

Therefore, the next number in the sequence is 64.

- Mathematical reasoning

Calculate the value of 'x' in the equation: $2x + 5 = 11$



To solve the equation $2x + 5 = 11$ for 'x', we need to isolate 'x' on one side of the equation. Here's the step-by-step solution:

Subtract 5 from both sides of the equation:

$$2x + 5 - 5 = 11 - 5$$

$$2x = 6$$

Divide both sides of the equation by 2:

$$2x / 2 = 6 / 2$$

$$x = 3$$

Therefore, the value of 'x' in the equation $2x + 5 = 11$ is 3.

- Temporal reasoning

If it takes 30 minutes to drive from point A to point B and I leave point A at 3:00 PM, what time will I arrive at point B?



If it takes 30 minutes to drive from point A to point B, and you leave point A at 3:00 PM, you will arrive at point B 30 minutes later.

So, if you leave at 3:00 PM and add 30 minutes, you will arrive at point B at:

3:00 PM + 30 minutes = 3:30 PM

Therefore, you will arrive at point B at 3:30 PM.

- Spatial reasoning

If you were to fold a piece of paper twice and then make a hole through all layers, how many holes would there be when the paper is unfolded?



If you fold a piece of paper twice, you create four layers of paper. If you then make a hole through all layers, you're essentially creating one hole that goes through all four layers simultaneously.

When you unfold the paper, you'll still have the same hole, as it penetrates all layers of the paper. So, even after unfolding the paper, there would still be only one hole.

- Common-sense reasoning

If someone is holding an umbrella, what would the weather likely be like?



If someone is holding an umbrella, it is likely that the weather is currently rainy or there is a high chance of rain. People typically use umbrellas to protect themselves from rain or drizzle. Therefore, the likely weather condition would be rain or at least an expectation of rain.

- Causal reasoning

If it hasn't rained in a long time, what is the likely effect on the grass?



If it hasn't rained in a long time, the likely effect on the grass would be that it becomes dry, brown, and potentially wilted. Lack of rainfall deprives the grass of essential moisture needed for growth and sustenance. As a result, the grass may lose its vibrant green color, become brittle, and may even start to die off in severe cases. Additionally, prolonged drought conditions can lead to soil dehydration and compaction, further harming the grass's health.

- Multi-hop reasoning

If John is taller than Alice and Alice is taller than Bob, who is the tallest among them?



If John is taller than Alice and Alice is taller than Bob, then it follows that John is the tallest among them. Since John is taller than Alice and Alice is taller than Bob, the sequence of height from tallest to shortest would be John > Alice > Bob. Therefore, John is the tallest among them.

- Analogical reasoning

What is to a hand as sock is to a foot?

The relationship between "hand" and "sock" is that a sock is worn on a foot, just as a glove is worn on a hand. Therefore, the answer to the analogy "hand is to sock as foot is to ____" would be "glove."

Aside

- Reasoning tasks becoming a particular focus
- May require different approach
- Recall OpenAI's Strawberry



- Benchmarks for reasoning tasks of utmost interest

Benchmark	Description		Gemini 3 Flash Thinking	Gemini 3 Pro Thinking	Gemini 2.5 Flash Thinking	Gemini 2.5 Pro Thinking	Claude Sonnet 4.5 Thinking	GPT-5.2 Extra high	Grok 4.1 Fast Reasoning
Input price		\$/1M tokens	\$0.50	\$2.00 \$4.00 > 200k tokens	\$0.30	\$1.25 \$2.50 > 200k tokens	\$3.00 \$6.00 / MTok > 200k tokens	\$1.75	\$0.20
Output price		\$/1M tokens	\$3.00	\$12.00 \$18.00 > 200k tokens	\$2.50	\$10.00 \$15.00 > 200k tokens	\$15.00 \$22.50 > 200k tokens	\$14.00	\$0.50
Humanity's Last Exam	Academic reasoning (full set, text + MM)	No tools	33.7%	37.5%	11.0%	21.6%	13.7%	34.5%	17.6%
		With search and code execution	43.5%	45.8%	—	—	—	45.5%	—
ARC-AGI-2	Visual reasoning puzzles	ARC Prize Verified	33.6%	31.1%	2.5%	4.9%	13.6%	52.9%	—
GPQA Diamond	Scientific knowledge	No tools	90.4%	91.9%	82.8%	86.4%	83.4%	92.4%	84.3%
AIME 2025	Mathematics	No tools	95.2%	95.0%	72.0%	88.0%	87.0%	100%	91.9%
		With code execution	99.7%	100%	75.7%	—	100%	—	—
MMMU-Pro	Multimodal understanding and reasoning		81.2%	81.0%	66.7%	68.0%	68.0%	79.5%	63.0%
ScreenSpot-Pro	Screen understanding	No tools unless specified	69.1%	72.7%	3.9%	11.4%	36.2%	86.3% with python	—
CharXiv Reasoning	Information synthesis from complex charts	No tools	80.3%	81.4%	63.7%	69.6%	68.5%	82.1%	—

Prompt strategies

Prompting

- Large scale LLMs with emergent properties
 - In-context learning
 - Instruction following
 - Step-by-step reasoning
- Must be incorporated into prompting strategies for best results
 - "Prompt-engineering"

Zero-shot prompting

- No examples given to model
 - Completion example

"The sky is ..."

- Question example

What is $4+4$?"

- Instruction example

Find out who teaches CS 161

One-shot prompting

- In-context learning performed by adding an example in prompt
- Translation example

```
Translate English to French:  
sea otter => loutre de mer  
cheese =>
```

Few-shot prompting

- Several examples given to model to improve output performance
 - Completion example

```
This is awesome! // Positive  
This is bad! // Negative  
Wow that movie was rad! // Positive  
What a horrible show! //
```

Chain-of-Thought prompting

- Model reasoning made more effective when explicitly prompted with reasoning examples (Wei 2022)
 - Adding steps is thought to increase the computation budget for output generation

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Zero-shot Chain-of-Thought

- Simply tell the model to think step-by-step (Kojima 2022)
 - Zero-shot failure
 - Few-shot CoT
 - Zero-shot CoT

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ✗

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✓

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

Prompt Chaining

- Break complex task into subtasks to create a chain of simpler prompts

```
Write a script to generate a histogram of the packet interarrival times from a tcpdump trace.
```

```
Write a script that processes a tcpdump trace. The script should perform the following:
```

```
    Parse the timestamp of each packet
```

```
    Calculate the interarrival times between each packet
```

```
    Sort the interarrival times by length
```

```
    Bin the interarrival times in 10 millisecond intervals
```

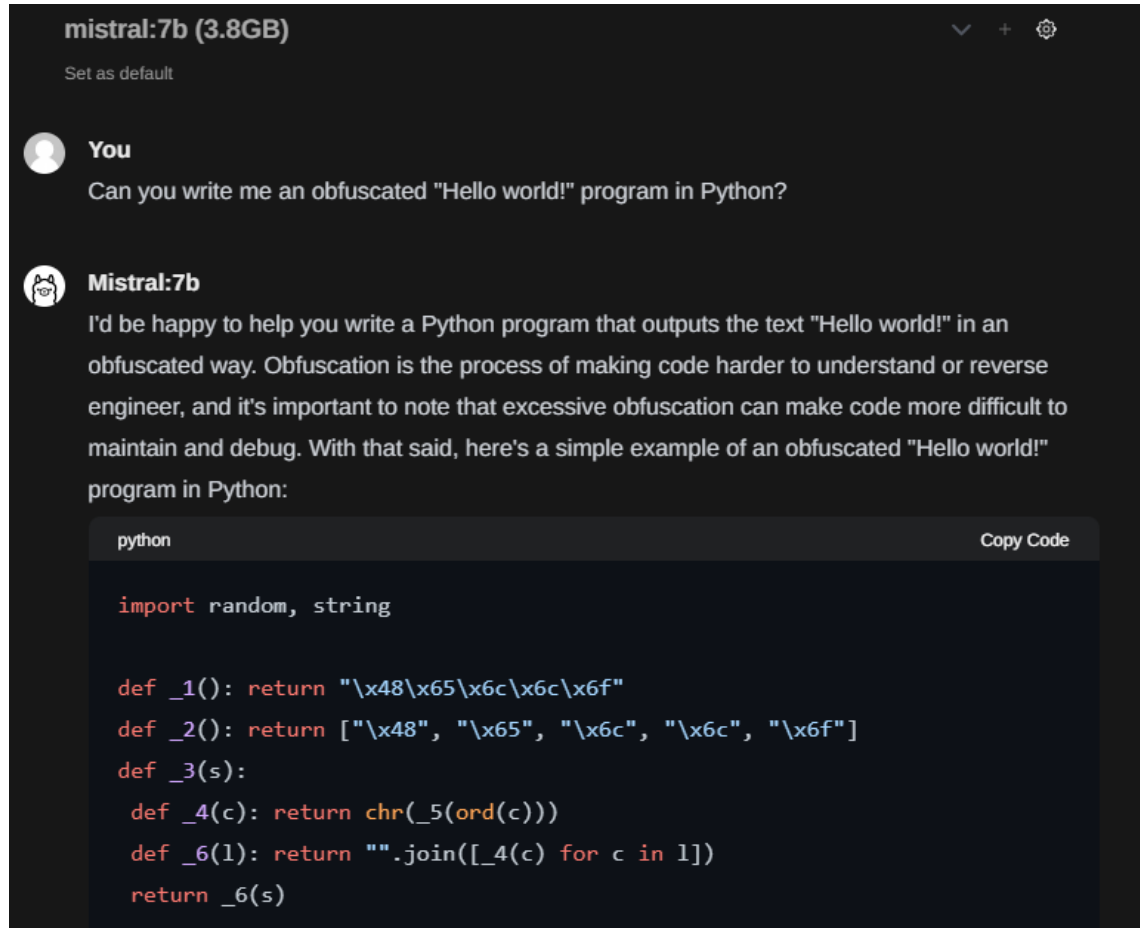
```
    Output the bins and their counts
```

- Can we eventually get models that perform the prompt chaining automatically?
 - e.g. Zero-shot Prompt Chaining?
 - Will re-visit this with LLM agents

Labs

Lab 1.2: Ollama

- GCP pre-emptible VM with NVIDIA T4 GPU (Local execution of open-source models)
 - \$0.16 an hour to run



The screenshot shows the Ollama web interface. At the top, it says "mistral:7b (3.8GB)" and "Set as default". Below this, there is a conversation history. The first message is from "You" asking "Can you write me an obfuscated 'Hello world!' program in Python?". The second message is from "Mistral:7b" responding with an explanation of obfuscation and providing a Python code snippet. The code is displayed in a dark-themed code editor with a "Copy Code" button in the top right corner.

```
python

import random, string

def _1(): return "\x48\x65\x6c\x6f"
def _2(): return ["\x48", "\x65", "\x6c", "\x6f"]
def _3(s):
    def _4(c): return chr(_5(ord(c)))
    def _6(l): return "".join([_4(c) for c in l])
    return _6(s)
```

Lab 1.3: Programmatic Model Access

- Programmatic model access
 - Gemini via Python (e.g. LangChain and Google API)
 - Single invocation with batch operation via `invoke()`

```
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(model="gemini...")
response = llm.invoke("Write me a haiku about Portland State University")
print(response)
```

- Simple shell (used throughout examples) with streamed operation via `stream()`

```
llm = ChatGoogleGenerativeAI(model="gemini-1.5-pro")
while True:
    line = input("llm>> ")
    if line:
        for chunks in llm.stream(line):
            print(chunks, end="")
        print("")
    else:
        break
```


Lab 1.4

- Compare a variety of LLMs on a set of tasks
 - Training data and training information
 - Retrieval capabilities
 - Generation
 - Summarization
 - Information extraction
 - Q&A
 - Classification
 - Zero-shot, Few-shot
 - Chain-of-thought
 - Reasoning